

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Equipping LLMs with active access to external knowledge is broadly important for tasks whose answers depend on information beyond parametric memory. Existing approaches either rely on prompting to elicit agentic behavior or encounter performance ceilings and collapse when RL is applied to interactive, long-horizon settings, leaving much of this capability underdeveloped. We introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an RL algorithm for training search-and-reasoning agents through sequence-level optimization and dynamic sample filtering. Using RL alone, our model learns to interleave multi-turn search with reasoning without supervised demonstration data. Across multiple QA benchmarks spanning single-hop, multi-hop, and long-tail factual settings, our 7B model improves over a comparable prior 7B system by **34.1%**; on complex multihop QA such as HotpotQA, it also exceeds the prior 14B model by nearly **9% relative** while maintaining stable training.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) now perform strongly across many specialized settings, including mathematics (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). Yet a general limitation remains: parametric knowledge is static and bounded by the data used during pretraining. A widely applicable way to relax this limitation is to give LLMs search access, turning them into agents that can query external knowledge sources when the task requires information not reliably stored in the model (Jin et al., 2025b). This setting is a representative form of tool-calling (Schick et al., 2023), in which the model must decide how to interact with an external search tool rather than merely generate from memory. Learning when to search, how to formulate queries, and how to use retrieved evidence defines a multi-step policy, making agentic search a natural sequential decision-making problem for Reinforcement Learning (RL).

Compared with Supervised Fine-Tuning (SFT), which depends on fixed demonstrations and provides little opportunity for exploration, RL offers a general framework for learning interactive policies from trial-and-error feedback (Chu et al., 2025). Value-free methods such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024) are therefore attractive for LLM training because they reduce memory overhead and avoid a separate value model. However, when these methods are used beyond constrained generation tasks and applied to interactive search, important instabilities become visible (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). Two issues are especially relevant across sparse-reward, long-horizon tasks. First, as identified by Zheng et al. (2025), GRPO optimizes token-level ratios while the reward is assigned to the sequence, producing high-variance gradients that can destabilize training. Second, sparse terminal rewards often create rollout groups with homogeneous outcomes, such as all successes or all failures, so the group-relative advantage collapses and provides little or no learning signal (Yu et al., 2025; Liu et al., 2025).

To address these paired challenges of instability and inefficient learning, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. The algorithm combines and adapts principles from recent policy-optimization work for settings where actions unfold over multiple reasoning and tool-use steps. DSPO adopts sequence-level optimization from GSPO (Zheng et al., 2025), matching the optimization unit to the sequence-level reward and thereby stabilizing updates for long trajectories. It also incorporates a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025), constructing training batches from rollout groups that contain both successful and unsuccessful outcomes for each prompt. This keeps the advantage signal $\hat{A}_i$ informative under sparse rewards. Together, these components form a coherent training framework for multi-turn search and reasoning, and they are relevant to broader sparse-reward agent learning scenarios with sequence-level feedback. Empirically, our model achieves a **34.1% relative**

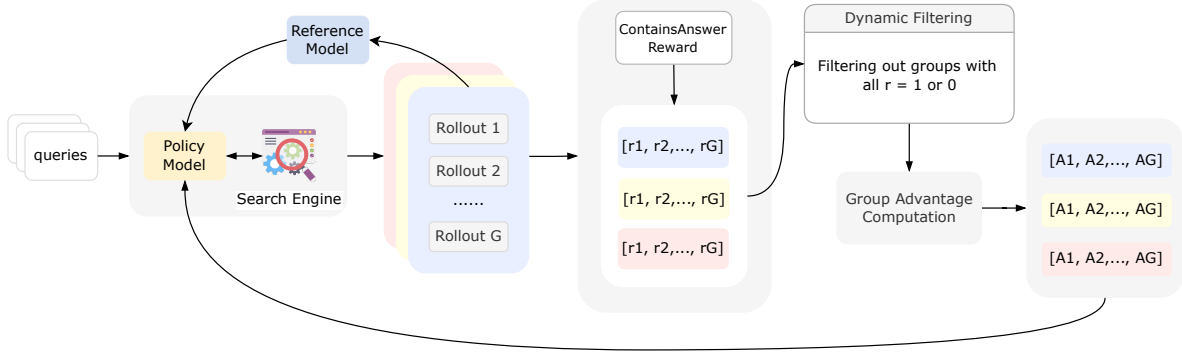


Figure 1: Overview of the DSPO training loop. For a given query, the policy generates a group of G trajectories through interaction with the search environment, and each trajectory receives a sparse terminal reward. The **dynamic filter** removes groups with homogeneous outcomes and continues sampling until the batch is filled, so each update contains an informative advantage signal. The resulting advantages update the policy through a sequence-level objective.

improvement over a leading 7B baseline (Jin et al., 2025b) and surpasses its 14B counterpart (Jin et al., 2025a) on complex multi-hop benchmarks such as HotpotQA by **nearly 9% relative** (0.613 vs. 0.563).

In summary, our main contributions are as follows:

- We propose DSPO, an RL algorithm for training agentic search models under sparse, sequence-level feedback. It unifies two broadly useful principles in one framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for maintaining an effective learning signal.
- We demonstrate DSPO’s performance gains on a benchmark suite that covers diverse QA conditions. Our 7B model achieves a **34.1% relative improvement** over a comparable 7B baseline and outperforms its 14B counterpart on complex multi-hop QA, with a nearly **9% relative gain** on HotpotQA (0.613 vs. 0.563).
- We provide empirical evidence that DSPO improves training stability across our evaluated conditions. Because these results use a basic BM25 retriever, the observed gains are attributable to the policy-optimization algorithm rather than a stronger retrieval system.

2 RELATED WORK

2.1 RL FOR LLMs

RL for LLMs has developed from Reinforcement Learning from Human Feedback (RLHF) methods based on PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) toward simpler direct-optimization frameworks such as DPO (Rafailov et al., 2023). Group Relative Policy Optimization (GRPO) (Shao et al., 2024) represents an important move toward value-free optimization by deriving advantages from group statistics rather than a learned value model. At the same time, its token-level objective can be unstable (Liu et al., 2025; Cui et al., 2025), motivating improvements that address different parts of the problem. GSPO uses a sequence-level objective aligned with sequence-level rewards (Zheng et al., 2025), DAPO improves learning under sparse rewards through dynamic outcome-based sampling (Yu et al., 2025), and GMPO uses geometric-mean aggregation to reduce sensitivity to token-level outliers (Zhao et al., 2025). In our agentic search experiments, however, these approaches still exhibit collapse or performance bottlenecks. DSPO builds on this broader line of work by combining sequence-level optimization with dynamic filtering and filling, targeting the general difficulty of stable policy learning for autonomous agents with sparse terminal feedback.

2.2 LLMs WITH AGENTIC RETRIEVAL

To mitigate the static knowledge limitations of LLMs, retrieval-augmented generation (RAG) connects models to external retrievers that can supply task-relevant and changing information (Lewis et al., 2020; Gao et al., 2023). Classical RAG systems typically retrieve documents with dense retrievers and concatenate them into the model input for generation (Karpukhin et al., 2020). Although effective in many settings, fixed retrieval pipelines provide limited autonomy

when a task requires iterative evidence gathering, query revision, or multi-step verification. Recent work therefore moves toward agentic paradigms in which LLMs plan, search, and reason over multiple turns. ReAct combines reasoning and acting for tool use in settings such as web navigation (Yao et al., 2023), while multi-agent systems such as AutoGen support collaborative workflows (Wu et al., 2024). Other agentic RAG and RL-based approaches use decision-making to improve retrieval and reasoning; for example, Wu et al. (2025) introduce Agentic Reasoning, and several RL-integrated methods (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) train LLMs to interleave reasoning and search without supervised traces. This end-to-end paradigm can internalize agent behavior while reducing framework engineering, but it still faces instability and performance limitations in open-ended search. DSPO addresses these bottlenecks by providing a stable policy-optimization framework for learning multi-turn search strategies.

3 METHODOLOGY

This section formulates agentic search as an RL problem and reviews policy-optimization methods that expose the broader challenges of sparse, sequence-level feedback. We then present **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, describing how its components improve stability and sample efficiency in this setting. We conclude with the integrated training algorithm used in our experiments.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. RL training for LLMs commonly relies on policy-gradient algorithms such as PPO (Schulman et al., 2017), which is widely used in alignment and other feedback-driven optimization settings. PPO updates a policy π_θ by maximizing a clipped surrogate objective on samples generated by an older policy $\pi_{\theta_{\text{old}}}$. With token-level averaging, the objective is:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. PPO also requires a separately trained value model to estimate token-level advantages $\hat{A}_t$ through Generalized Advantage Estimation (GAE) (Schulman et al., 2015), which increases memory cost and can introduce another source of instability.

GRPO (Shao et al., 2024) removes this value-model requirement by sampling a group of G responses $\{y_i\}_{i=1}^G$ for a prompt x and normalizing each response reward against group statistics. This design is appealing for many LLM RL workloads, but, like PPO, the resulting objective is still optimized at the token level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

Thus, every token in response y_i receives the same advantage $\hat{A}_i$, even though that advantage is derived from a sequence-level reward.

Agentic Search as a Markov Decision Process. We formalize agentic search and reasoning as a sequential decision-making problem, using a discrete-time, finite-horizon Markov Decision Process (MDP) that captures a broad class of tool-mediated interactions.

- **State (s_t):** At turn t , the state s_t contains the full interaction history: the initial question q , the model’s prior thoughts and search queries, and the evidence returned by the environment. Because retrieved information must be interpreted before later decisions are made, the state grows with the trajectory and creates long-horizon dependencies that are difficult for conventional token-level RL to optimize.

- **Action** (a_t): An action is a full textual segment generated by the policy π_θ , combining free-form reasoning with a decision about whether to request external information or answer. The action ends either with a `</tool_call>` token, which triggers a search and places the returned results within `</tool_response>`, or with a `</answer>` token, which terminates the trajectory. This structured action space requires the agent to learn both *what* to generate and *when* to search, producing variable-length trajectories.
- **Policy** (π_θ): The policy π_θ is the underlying LLM, which generates tokens autoregressively conditioned on the current state. Although the policy can condition on the full history, the optimization target is computed only on model-generated thoughts and actions, with retrieved environmental content masked out (Jin et al., 2025b). Optimizing this policy therefore requires credit assignment across long sequences in which intermediate reasoning steps are not directly supervised, motivating stable sequence-level optimization.
- **Trajectory** (τ): A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records the complete reasoning and tool-interaction process for a question, including model-generated actions and search results returned by the environment. Since the final reward is assigned to the whole trajectory, early query and reasoning decisions can affect eventual answer correctness.
- **Reward** ($R(\tau)$): We use a sparse terminal reward that reflects final-answer correctness: a trajectory receives $R = 1$ if the final answer contains the ground-truth text and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we treat the search engine $\mathcal{S}$ as part of the environment rather than as a fixed preprocessing module. The policy LLM π_θ learns trajectories that interleave reasoning with calls to $\mathcal{S}$. The optimization goal is to maximize expected reward while using a KL term to keep the learned policy close to a reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ indicates that trajectory generation depends jointly on the policy’s token generation and the information returned by the search engine $\mathcal{S}$.

Motivation for DSPO. Frameworks such as Search-R1 (Jin et al., 2025b) show that agentic search can be cast as an RL problem, but conventional algorithms such as PPO or GRPO remain difficult to apply reliably in this broader interactive setting. Open-ended search amplifies the instability of token-level optimization because the reward is assigned to an entire trajectory while updates are computed at individual token positions (Zheng et al., 2025). This mismatch can produce high-variance gradients that accumulate over long trajectories and lead to policy collapse. Sparse binary rewards introduce a second problem: many sampled groups contain only successful or only unsuccessful trajectories, resulting in abnormal advantages and little useful gradient signal (Yu et al., 2025; Liu et al., 2025). DSPO is designed to address both failure modes together.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO introduces two mechanisms that are applicable to this class of sparse-reward sequence problems: (1) policy optimization at the sequence level, aligning the training objective with trajectory-level rewards, and (2) dynamic filtering that keeps each training batch informative by requiring a non-zero learning signal. Figure 1 depicts the integrated training process.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Following the sequence-level perspective of GSPO (Zheng et al., 2025), DSPO replaces token-level importance ratios with a sequence-level counterpart better matched to terminal rewards. For response y_i , the sequence-level importance ratio $s_i(\theta)$ is defined as the geometric mean of token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization reduces variance and keeps $s_i(\theta)$ in a comparable numerical range across different sequence lengths, which supports stable clipping in variable-length trajectories.

Gradient Analysis. The gradient comparison explains why DSPO improves stability in sequence-reward settings. The gradient of the token-level GRPO objective (unclipped) weights each token’s log-probability gradient by a token-specific ratio $r_{i,t}(\theta)$, which can be noisy along long trajectories. In contrast, the DSPO sequence-level objective weights the average log-probability gradient of the whole sequence by a single sequence-level factor $s_i(\theta)$:

$$\nabla_{\theta} J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_{\theta} J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (8)$$

By applying one correction factor to the entire trajectory, DSPO reduces the accumulation of token-level noise that can affect prior methods and yields more stable training. In parallel, dynamic filtering keeps the advantage signal $\hat{A}_i$ well defined by constructing batches from rollout groups that contain both successes and failures, reducing wasted samples in sparse-reward environments.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The sparse binary reward used in our experiments highlights a common challenge for group-based advantage estimation. If all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined. These homogeneous groups provide little useful gradient information and waste computation.

To address this issue, DSPO uses a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, it retains only trajectory groups with both successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training only when its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This condition ensures non-zero reward variance inside each training group and therefore a meaningful advantage signal. The dynamic selection process curates informative samples for each policy update, converting sparse terminal feedback into a denser and more efficient learning signal.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

Combining sequence-level ratios with dynamic filtering yields the final DSPO objective. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, we compute $\hat{A}_i$ with group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_{θ} is updated by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of token-level ratios. During likelihood calculation, we follow Jin et al. (2025b) and mask all tokens retrieved from the search tool, so the model is trained to use external knowledge for reasoning rather than to reproduce retrieved text. Algorithm 1 gives the full training procedure.

4 EXPERIMENTS

This section evaluates Dynamic-filter Sequence-level Policy Optimization (DSPO) under the same datasets, retrievers, and evaluation conditions used for our comparisons. The experiments test whether DSPO (1) improves performance across a diverse QA benchmark suite, (2) stabilizes RL training in interactive search, and (3) obtains its gains from the combination of sequence-level optimization and dynamic filtering rather than from either component alone.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```



Figure 2: **Validation performance of DSPO across seven benchmarks during training.** Accuracy increases steadily across the evaluated QA suite, indicating that reward improvement transfers to held-out search-and-reasoning benchmarks rather than remaining confined to the training signal.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), we use the prompt template in Table 1 to define the model’s action format during search, including `<think>`, `<tool_call>`, and `<answer>`.

Benchmarks and Baselines. Our evaluation follows the established Search-R1 protocol (Jin et al., 2025b) so that comparisons reflect algorithmic differences rather than changes in task design. We train on a composite dataset containing the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018). We then evaluate on seven QA test sets that cover factual, long-tail, and multi-hop question answering: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

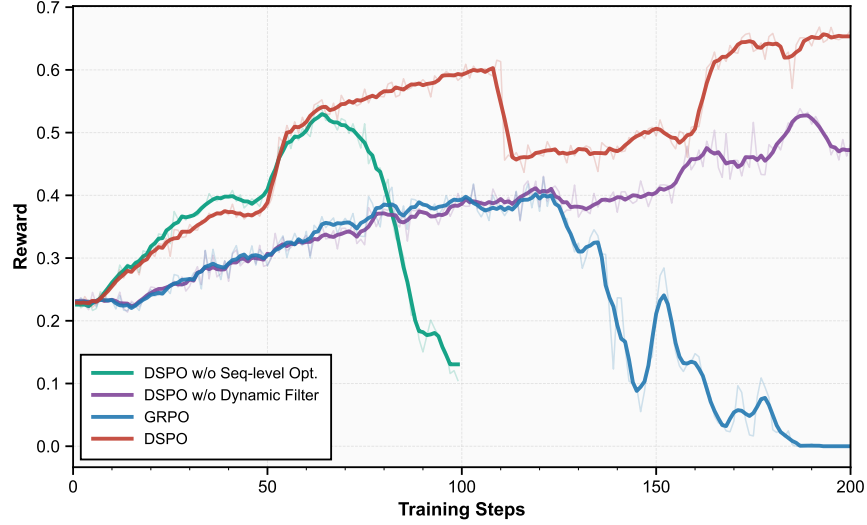


Figure 3: **Training reward dynamics of DSPO and its ablations.** Comparative learning curves show that DSPO (red) improves stably and monotonically. Token-level variants (green, blue) collapse early in training, while the sequence-level variant without dynamic filtering (purple) remains stable but plateaus below the full method.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: Prompt template used to standardize reasoning, search calls, and final-answer formatting in our experiments.

The comparison suite combines external baselines with internal ablations to separate overall performance from component-level effects. External baselines are Qwen2.5-7B and 14B models trained with PPO and GRPO in the Search-R1 framework (Jin et al., 2025b;a). For internal analysis, we include two ablations: (1) **DSPO w/o dynamic filter**, equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reduces the method to the strong token-level policy used in DAPO (Yu et al., 2025).

For implementation, all 7B training starts from Qwen2.5-7B-Instruct. The experiments are built on the VeRL framework (Sheng et al., 2025); we adapt its Search-R1-like example code and scripts to implement our method. Because our work uses a modified reward function, we retrain the PPO and GRPO Search-R1 baselines (Jin et al., 2025b;a) under the same experimental conditions, and these retrained models serve as the primary external comparisons.

Implementation and Evaluation. To isolate the contribution of the policy-optimization algorithm, every RL experiment uses the same standard BM25 retriever. This controlled retrieval setting keeps the external knowledge interface fixed, so performance differences reflect the learned policy. All methods are trained with a sparse binary reward based on substring Exact Match (subEM) of the final answer, and subEM is also the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 reports the main comparison and ablation study together, because DSPO is designed as a combination of complementary mechanisms. Presenting external baselines beside ablated variants shows both the method’s overall performance across the QA suite and the extent to which each component contributes under the same evaluation protocol.

Table 2: Comparison of DSPO, Search-R1 baselines, and ablation variants across seven QA benchmarks. Baselines include 7B and 14B Search-R1 models trained with GRPO and PPO (Jin et al., 2025b;a); ablations remove either dynamic filtering or sequence-level optimization. Original Search-R1 EM scores appear in parentheses, while our retrained results use the adjusted reward for consistent evaluation. Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	0.754
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

Comparison with Baselines. Table 2 shows that DSPO improves performance across the evaluated QA benchmarks. The DSPO-trained 7B agent reaches an average score of 0.531, a 34.1% relative improvement over the same-sized Search-R1 GRPO baseline. The 7B DSPO model also slightly exceeds the average score of the larger Search-R1 14B models trained with GRPO and PPO, suggesting that the gains come from more effective and stable policy learning rather than from model scale alone.

Analysis of Ablations. The ablations in Table 2 show that both DSPO components matter under the evaluated conditions. Removing the dynamic filter (‘w/o dynamic filter’) reduces the average score to 0.313, indicating that sequence-level updates alone are not sufficient when homogeneous-reward batches provide weak advantage signals. Removing sequence-level optimization (‘w/o sequence-level opt.’) yields an average score of 0.406; this token-level variant performs better than the filter-less variant but remains far below full DSPO and, as shown next, is prone to collapse. These results support the intended division of labor: sequence-level updates stabilize optimization, while dynamic filtering converts sparse rewards into more useful training signal.

Beyond aggregate metrics, DSPO produces qualitatively useful search behaviors, including recognition of irrelevant results, query reformulation, and multi-turn verification (see Appendix A for detailed examples). These behaviors emerge from RL training with DSPO rather than from supervised search demonstrations.

4.3 ANALYSIS OF TRAINING DYNAMICS

To examine stability and efficiency during optimization, we compare the training reward dynamics of DSPO, its ablations, and key baselines. Figure 3 visualizes these dynamics and clarifies how the two components affect learning behavior.

DSPO (red) shows a smooth, monotonic increase and converges to the highest reward level among the compared methods. This behavior is consistent with the stabilizing effect of sequence-level updates. In contrast, the **token-level methods**—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—collapse early after a short improvement phase, matching the expected failure mode of high-variance token-level gradients. **DSPO w/o Dynamic Filter** (purple) remains stable because it uses sequence-level updates, but it plateaus at a lower reward because it lacks consistently informative filtered groups. These dynamics illustrate why DSPO requires both sequence-level stability and dynamic filtering.

To check whether higher training reward corresponds to held-out task performance rather than reward hacking, we track validation accuracy throughout training. Figure 2 shows that DSPO improves consistently on NQ, HotpotQA, and the other evaluated benchmarks, broadly following the stable reward curve. This pattern indicates that the learned behavior transfers across the tested search-and-reasoning benchmarks.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

We further examine whether the same optimization principles hold across model scale and beyond the primary agentic-search benchmark suite.

Scalability to Larger Models. We test whether DSPO’s stability benefits extend to a larger model by training Qwen2.5-14B-Instruct. As shown in Table 3, the DSPO-trained 14B model reaches an average accuracy of 60.6%, outperforming the GRPO-14B baseline (53.0%) by a relative margin of **14.3%**. These results indicate that DSPO can make effective use of additional model capacity under the same benchmark and retrieval conditions.

Table 3: Evaluation of DSPO at the Qwen2.5-14B-Instruct scale under the same QA benchmark protocol. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. We also test whether the same optimization design applies outside multi-turn search by evaluating single-turn mathematical reasoning with Qwen2.5 and Qwen3 models. Table 4 reports results on Math500 and Olympiad-Bench. DSPO outperforms GRPO for both the 7B and 4B settings, suggesting that the sequence-level and filtering principles can help beyond agentic retrieval when tasks still involve sparse outcome feedback.

Table 4: Generalization check on single-turn mathematical reasoning benchmarks. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

This work addresses instability and sample inefficiency in RL training for autonomous LLM search agents, a setting that exemplifies broader sparse-reward, sequence-level agent learning. We introduced Dynamic-filter Sequence-level Policy Optimization (DSPO), which combines sequence-level optimization to reduce policy-collapse risk with dynamic outcome-based filtering to keep sparse rewards informative. Across the evaluated question-answering benchmarks, DSPO improves performance while showing more stable training dynamics than the compared methods.

By learning from environmental feedback alone, DSPO provides a practical route for building LLM agents without costly expert demonstrations. The same framework can be explored with stronger retrievers, richer tool interfaces, or more complex multi-tool tasks. Because sparse rewards and unstable policy gradients also arise outside search, the principles behind DSPO may be useful in domains such as mathematics and code generation; our mathematical reasoning results provide an initial indication, while broader validation remains future work. More generally, matching the optimization unit to the reward signal and maintaining signal density are promising design principles for reliable autonomous AI systems.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

To illustrate the search strategies learned under DSPO, Table A.1 presents representative model trajectories. The examples show behaviors relevant across multi-turn retrieval tasks: reformulating queries after irrelevant results, verifying information through follow-up searches, and extracting answers from retrieved evidence.

Table A.1: Representative trajectories illustrating learned search-and-reasoning behavior. The examples show (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies that are useful across multi-turn retrieval settings.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let's try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin's occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician